

01 — Malware Analysis

Static, dynamic, and C2 recognition for DFIR exams

Educational / Defensive DFIR Study Notes

LEARNING OBJECTIVES

- Apply a static triage workflow: strings → PE → entropy → FLOSS
- Interpret sandbox / VT / Hybrid Analysis outputs for IR decisions
- Recognize common C2 channel indicators without reverse-engineering every sample
- Avoid exam traps (fake compile times, entropy myths, AV label overtrust)

SCOPE REMINDER

- This material covers detection, artifacts, and incident response only.
- Attacker actions are phrased as 'adversaries may...' then pivot to evidence.
- Tool/command names appear as IOC strings / exam knowledge — not how-to guides.

Why analysts start static

Static analysis is fast, offline, and preserves the sample. It answers: What does this file claim to be? What strings and imports suggest capability? Is it packed? Dynamic analysis and sandboxes then confirm behavior. In IR you rarely need full RE — you need IOCs, family hints, and enough confidence to contain.

Strings hunting — what to extract

- **Network:** URLs, IPs, domains, User-Agent strings, API endpoints
- **Host:** file paths, registry keys, service names, mutexes, named pipes
- **Credentials / discovery:** default passwords (rare), LDAP paths, share names
- **Anti-analysis:** VM MAC prefixes, debugger strings, sandbox usernames

FLOSS (FireEye Labs Obfuscated String Solver) recovers strings that plain strings miss: stack strings, decoded obfuscated blobs, and tightly encoded constants. On exams, FLOSS is the answer when 'strings found nothing useful.'

PE header triage

Portable Executable structure tells you architecture, entry point, sections, and imports. Tools: file, readpe, Python pefile, objdump / dumpbin.

PE field	Analyst question	Exam note
Compile Time (TimeDateStamp)	When was it built?	Often fake / stamped — do not treat as ground truth
Sections (.text/.rdata/.data/.rsrc)	Odd names? Huge entropy section?	UPX0/UPX1, .packed, or single large section → packing hint
Imports	What APIs are used?	Few imports + LoadLibrary/GetProcAddress → dynamic resolution / packing
Entry Point	Does EP land in unexpected section?	EP outside .text is a packing/infection clue
Subsystem / Machine	GUI vs console? x86 vs x64?	Mismatch with alleged 'document' dropper is suspicious

Entropy

Shannon entropy near **7–8** suggests compressed or encrypted content. This is a **heuristic only**. Legitimate installers and resources can be high-entropy. Low entropy does not mean clean. Combine with section layout, imports, and sandbox behavior.

Dynamic analysis & reputation

Source	What you get	IR use
VirusTotal	Multi-AV score, family labels, community notes, basic indicators	Prioritize; never sole verdict — look at consistent family names
Hybrid Analysis	Behavioral report, network, dropped files, MITRE map	Quick capability overview for containment
Any.Run / Joe Sandbox	Interactive or deep sandbox traces	Confirm C2 hosts, persistence writes, process trees
IDA / Ghidra / x64dbg	Deep RE when needed	Reserve for novel malware or when IOCs insufficient

C2 channel recognition

Adversaries may use various channels to beacon and receive tasking. Analysts identify the **channel type** from network and host evidence:

Channel	Typical evidence	Why it matters
HTTP / HTTPS	Periodic POSTs; odd JA3/User-Agent; beacon intervals	Blends with web traffic; inspect proxy / SSL inspection logs
DNS	High query volume; long/encoded subdomains	Often allowed outbound when web is restricted

Channel	Typical evidence	Why it matters
Loopback	127.0.0.1 listeners; local proxying	May hide staging between local components
Named pipe	\\.\pipe\...; Sysmon pipe events	Lateral / local IPC; less visible on network sensors
Beacon pattern	Jittered regular intervals in netflow	Distinguish from human browsing burstiness

How do I prove X? — Analyst workflow

- **Suspected packed sample:** entropy + section anomalies + FLOSS recovers URLs → submit to sandbox → extract C2 → block IOC + hunt fleet
- **Suspected family:** VT consistent labels + YARA / unique mutex → confirm with sandbox process tree → search EDR for mutex/path
- **Suspected beacon:** netflow regularity + DNS/HTTP to rare domain + process that owns the socket (netscan / EDR) → isolate host

Common exam traps

- Treating PE TimeDateStamp as reliable timeline evidence (often forged)
- Declaring malware solely because entropy >7
- Trusting a single AV engine label over behavioral evidence
- Stopping at 'VirusTotal clean' — fresh or targeted malware may have zero detections
- Confusing HTTP 200 C2 responses with 'benign web browsing' without process context

QUICK REFERENCE

- Order: hashes → strings/FLOSS → PE/entropy → VT/sandbox → IOC hunt
- FLOSS = obfuscated/stack strings; plain strings = low hanging fruit
- C2: HTTP(S), DNS, loopback, named pipe, beacon timing
- Fake compile time; entropy is heuristic; VT is prioritization not truth

Mini Self-Check

Answer these without looking back. Answers are on the next page.

- Q1. When plain strings fails, which tool recovers obfuscated/stack strings?
- Q2. Why is PE compile time often unreliable for timelines?
- Q3. Entropy of 7.4 on a section — what can you conclude?
- Q4. Name three C2 channel types relevant to DFIR exams.
- Q5. What should you do if VT shows 0 detections but sandbox shows beaconing?

Self-Check Answers

A1. FLOSS (FireEye Labs Obfuscated String Solver).

A2. Adversaries (and some packers/tools) commonly forge or stamp TimeDateStamp; treat it as a weak hint only.

A3. It is a packing/encryption heuristic only — not proof of malware; correlate with imports, EP, and behavior.

A4. Any three of: HTTP, HTTPS, DNS, loopback, named pipe (plus beaconing as a pattern).

A5. Treat behavioral evidence as authoritative for IR: extract IOCs, contain, hunt; VT lag is common for novel samples.